

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационных систем

ПРАКТИЧЕСКАЯ РАБОТА № 3
по дисциплине «Методы и средства проектирования
информационных систем»
ТЕМА: ПК и СЕРВЕРА

Студент гр. 3376

Студент гр. 3376

Студентка гр. 3376

Преподаватель

Михайлов Н.Ф.

Константинов Р.И.

Дегтярева М.И.

Дюков Н.В.

Санкт-Петербург

2026

ЗАДАНИЕ

НА ПРАКТИЧЕСКУЮ РАБОТУ №3

Тема работы: ПК и Сервера

Исходные данные:

Создать классификатор изделий, который хранится в базе данных, и сервер, позволяющий получать доступ к БД посредством сервера API.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Анализ предметной области», «Разработка объектной модели этапа проектирования», «Разработка модели хранения», «Разработка базы данных», «Разработка основных процедур», «Результаты тестирования», «Заключение», «Список использованных источников», «Приложение».

Предполагаемый объем пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: 06.03.2026

Дата сдачи реферата:

Дата защиты реферата:

Преподаватель

Михайлов Н.Ф.

Константинов Р.И.

Дегтярева М.И.

Дюков Н.В.

АННОТАЦИЯ

В данной работе рассматривается расширение ранее разработанного классификатора изделий путём внедрения механизма справочников. Справочники используются для хранения структурированных наборов данных, описывающих параметры изделий различных типов, включая числовые значения и значения, заданные через перечисления.

В рамках работы выполнен анализ предметной области, связанной с ведением справочников изделий и их параметров. Определены требования к системе, обеспечивающей гибкое управление составом параметров, их наследованием и ограничениями значений. Разработана объектная модель этапа проектирования, позволяющая конструировать структуру справочника и управлять параметрами изделий.

Также разработана модель хранения данных в реляционной базе данных, включающая сущности справочников, параметров и их значений. Реализованы основные процедуры, обеспечивающие создание, редактирование и поиск данных в справочниках.

Проведено тестирование разработанных решений, подтверждающее корректность функционирования системы. Полученные результаты демонстрируют возможность гибкого и расширяемого управления параметрами изделий без изменения программного кода.

SUMMARY

In this work, the extension of a previously developed product classifier is considered through the implementation of a dictionary mechanism. Dictionaries are used to store structured datasets describing parameters of various product types, including both numerical values and values defined through enumerations.

As part of the work, an analysis of the domain related to managing product dictionaries and their parameters was carried out. Requirements for a system providing flexible management of parameter composition, their inheritance, and value constraints were defined. An object model for the design stage was developed, enabling the construction of dictionary structures and the management of product parameters.

A data storage model in a relational database was also developed, including entities for dictionaries, parameters, and their values. Core procedures were implemented to support the creation, editing, and retrieval of data within the dictionaries.

Testing of the developed solutions was conducted, confirming the correct functioning of the system. The obtained results demonstrate the possibility of flexible and extensible management of product parameters without modifying the program code.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	6
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ С ПРИМЕРАМИ ПЕРЕЧИСЛЕНИЙ.....	7
2. РАЗРАБОТКА ОБЪЕКТНОЙ МОДЕЛИ ЭТАПА ПРОЕКТИРОВАНИЯ..	9
3. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ.....	13
4. РАЗРАБОТКА БАЗЫ ДАННЫХ.....	17
5. РАЗРАБОТКА ОСНОВНЫХ ПРОЦЕДУР.....	20
6. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	21
ЗАКЛЮЧЕНИЕ.....	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	25

ВВЕДЕНИЕ

В современных информационных системах, предназначенных для хранения и обработки информации о продукции, требуют высокой гибкости и адаптивности. Это связано с необходимостью учитывать разнообразие характеристик продукции, которые могут изменяться в зависимости от предметной области.

В ранее разработанном классификаторе была реализована иерархическая структура категорий и поддержка параметров изделий, включающая перечисления. Однако данная модель ограничена в части расширяемости и управления сложными наборами параметров.

Для решения данной проблемы вводится понятие справочников, позволяющих описывать изделия с использованием настраиваемых параметров различных типов. Это даёт возможность динамически изменять структуру данных, добавлять новые характеристики и задавать ограничения без необходимости изменения программного кода.

Целью данной работы является разработка подсистемы справочников изделий, обеспечивающей хранение, редактирование и поиск данных с использованием гибкой модели параметров.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ С ПРИМЕРАМИ ПЕРЕЧИСЛЕНИЙ

В рассматриваемой предметной области система предназначена для хранения и обработки информации об изделиях, объединённых в классификатор. Каждое изделие характеризуется набором параметров, которые могут иметь различные типы: числовые, текстовые и значения из перечислений.

Для описания взаимодействия пользователей с системой была разработана диаграмма вариантов использования (Рисунок 1).

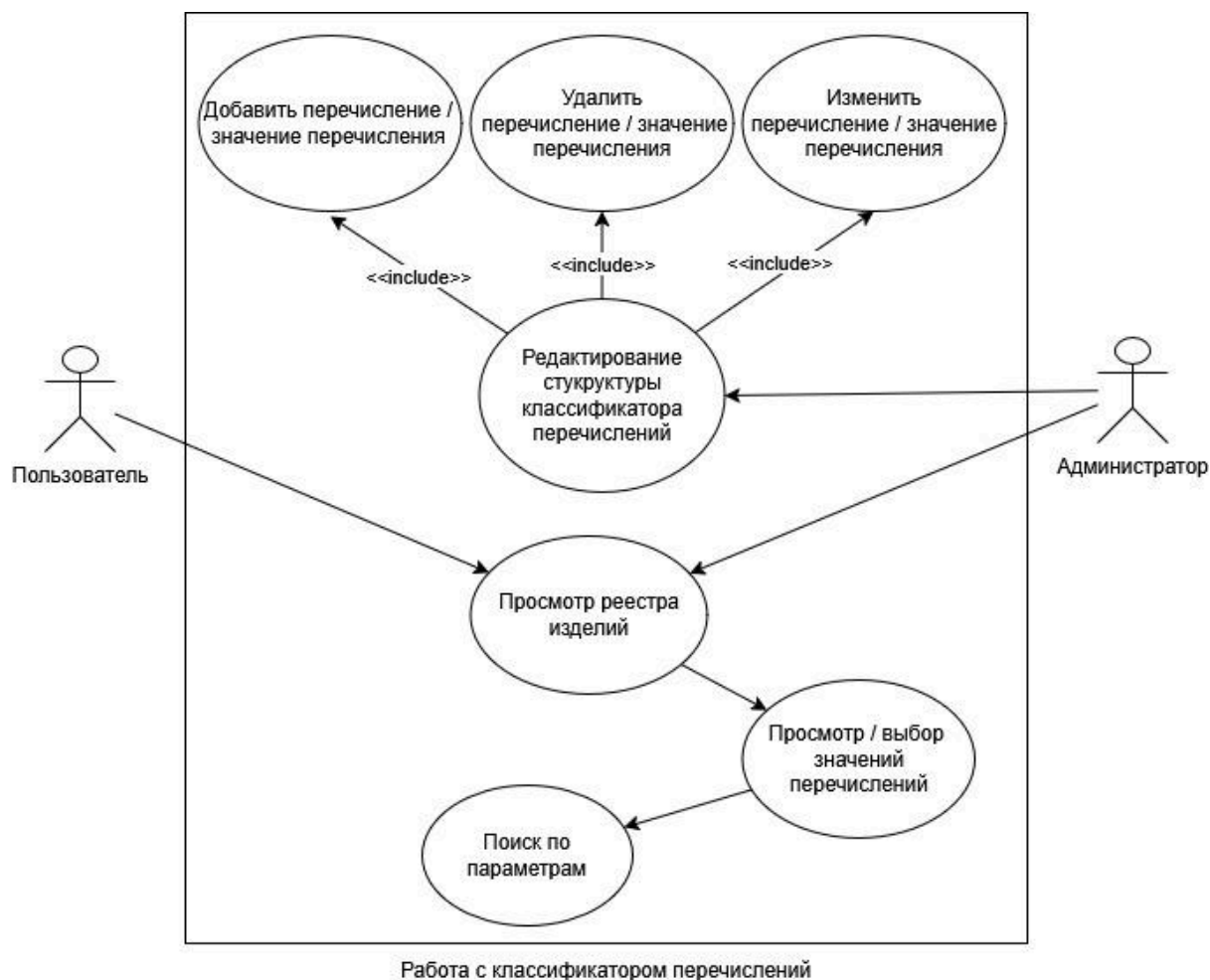


Рисунок 1 — use-case диаграмма

В системе выделяются следующие акторы:

- Пользователь — осуществляет просмотр данных классификатора, справочников и параметров изделий.
- Администратор — расширенный пользователь, управляющий структурой реестра.

Основные сценарии использования включают:

Для пользователя:

- Просмотр реестра изделий
- Просмотр/выбор значений перечислений
- Поиск по параметрам

Для администратора:

- Редактирование структуры классификатора перечислений
- Добавление перечислений и их значений
- Удаление перечислений и их значений
- Редактирование перечислений и их значений

2. РАЗРАБОТКА ОБЪЕКТНОЙ МОДЕЛИ ЭТАПА ПРОЕКТИРОВАНИЯ

На основе проведённого анализа была разработана объектная модель, обеспечивающая поддержку справочников и параметров изделий.

Диаграмма классов объектной модели представлена на рисунке 2.

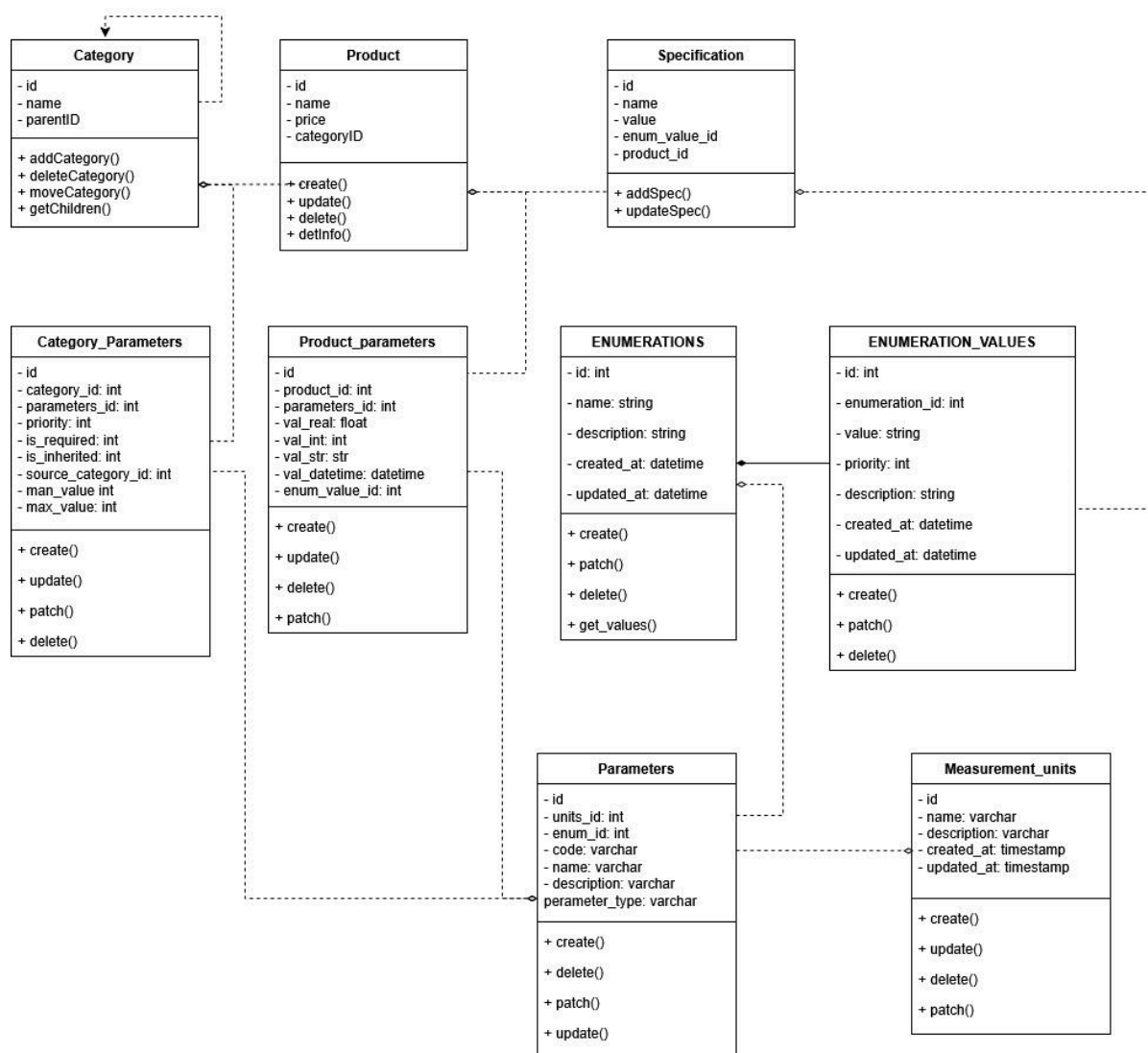


Рисунок 2 — Диаграмма классов объектной модели

Класс **Category_Parameters**

Предназначен для хранения параметров, связанных с конкретной

категорией классификатора. Он определяет, какие характеристики могут быть заданы для изделий данной категории.

Атрибуты:

- id — уникальный идентификатор
- category_id — FK
- parameters_id — FK
- priority — приоритет отображений
- is_required — обязательность
- is_inherited — наследуется ли
- source_category_id — откуда унаследован
- min_value — минимальное значение
- max_value — максимальное значение

Методы:

- create() — создание параметра категории
- update() — изменение параметра
- delete() — удаление параметра
- patch() — частичное обновление

Класс **Product_parameters**

Хранит параметры, назначенные конкретному изделию.
Обеспечивает связь между продуктом и его характеристиками.

Атрибуты:

- id — уникальный идентификатор
- product_id — FK
- parameters_id — FK

- val_real — вещественное значение
- val_int — целочисленное значение
- val_str — строковое значение
- val_datetime — значение даты и времени
- enum_value_id — значение перечисления

Методы:

- create() — добавление параметра изделию
- update() — изменение
- delete() — удаление
- patch() — частичное обновление

Класс **Parameters**

Представляет универсальную сущность параметра, используемую для описания характеристик изделий независимо от уровня (категория или продукт).

Атрибуты:

- id — идентификатор
- name — название параметра
- units_id — FK
- enum_id — FK
- code — код параметра
- description — описание параметра
- parameter_type — тип параметра

Методы:

- create() — создание параметра
- update() — обновление

- delete() — удаление
- patch() — частичное изменение

Класс **Measurement_units**

Предназначен для хранения хранения единиц измерения для числовых параметров.

Атрибуты:

- id — идентификатор
- name — название
- description — описание
- created_at — время создания
- updated_at — время обновления

Методы:

- create() — создание параметра
- update() — обновление
- delete() — удаление
- patch() — частичное изменение

Параметры, заданные для родительского класса, автоматически распространяются на дочерние классы. Также предусмотрена возможность задания ограничений для числовых параметров, включая минимальные и максимальные значения.

Разработанная объектная модель обеспечивает гибкость системы и возможность её дальнейшего расширения.

3. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ

Для реализации объектной модели была разработана модель хранения данных на основе реляционной базы данных (Рисунок 3).

Модель хранения включает следующие основные сущности:

- таблица справочников (классов изделий);
- таблица параметров;
- таблица значений параметров;
- таблицы перечислений (разработанные ранее);
- таблицы связей между изделиями и параметрами.

Каждый параметр связан с определённым классом изделия и может наследоваться дочерними классами. Значения параметров хранятся отдельно, что позволяет гибко управлять данными.

Для числовых параметров предусмотрены поля для хранения ограничений (минимум, максимум), что обеспечивает контроль корректности вводимых данных.

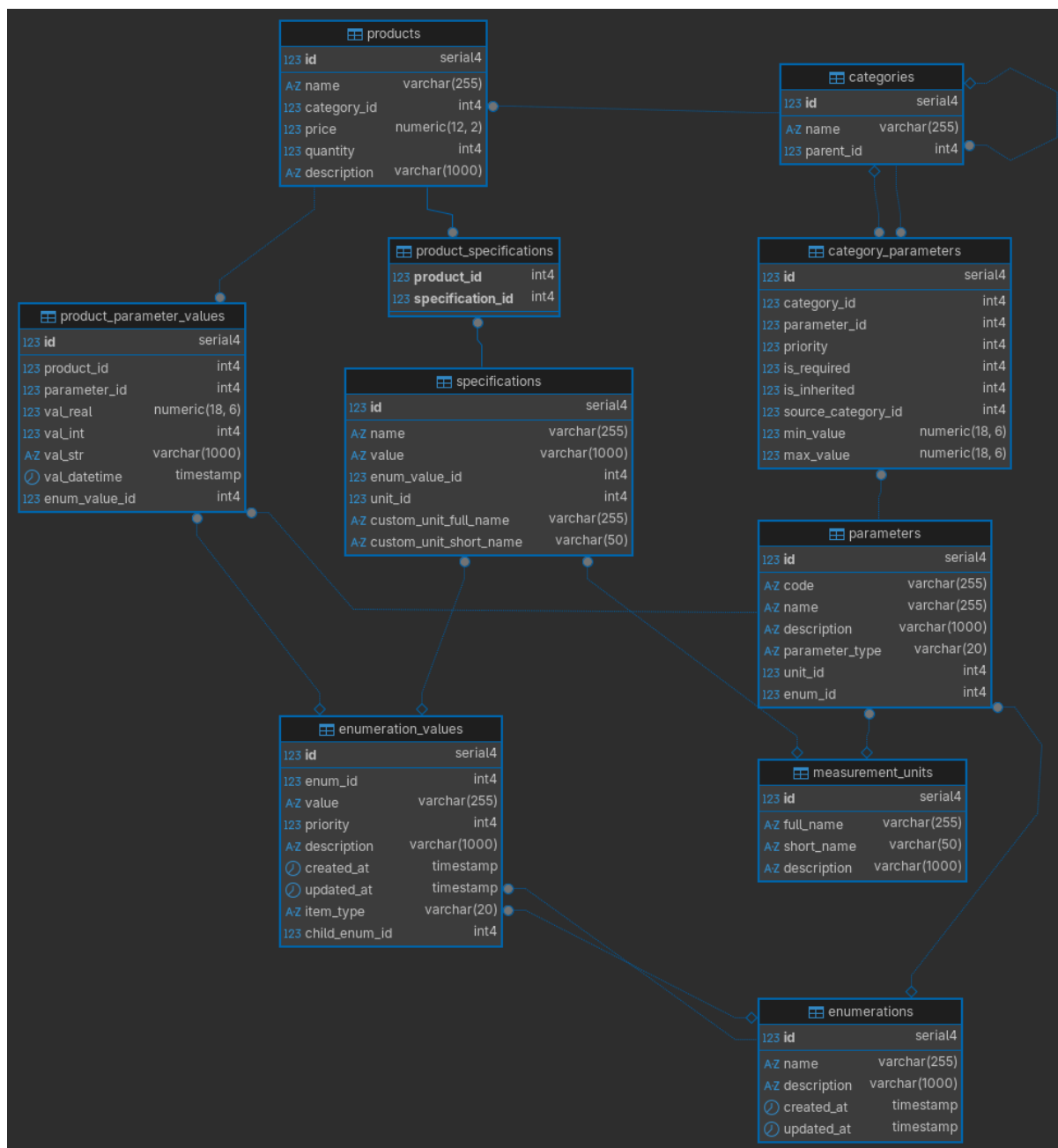


Рисунок 3 — диаграмма классов

Таблица **parameters**

Предназначена для хранения универсальных параметров, используемых при описании изделий.

Поля:

- id (PK) — уникальный идентификатор
- code — код параметра

- name — имя параметра
- description — описание
- parameter_type — тип (числовой, строковый, перечисление)
- unit_id (FK) — ссылка на единицу измерения
- enum_id (FK) — ссылка на перечисление

Таблица **measurement_units**

Хранит единицы измерения, применяемые к числовым параметрам.

Поля:

- id (PK) — идентификатор
- full_name — полное название
- short_name — сокращённое обозначение
- description — описание

Таблица **category_parameters**

Связующая таблица между категориями и параметрами.

Поля:

- id (PK)
- category_id (FK)
- parameter_id (FK)
- priority — порядок отображения
- is_required — обязательность
- is_inherited — наследуется ли параметр
- source_category_id — откуда унаследован
- min_value — минимальное значение
- max_value — максимальное значение

Таблица **product_parameter_values**

Хранит значения параметров конкретных изделий.

Поля:

- id (PK)
- product_id (FK)
- parameter_id (FK)
- val_real — числовое значение
- val_int — целое значение
- val_str — строковое значение
- val_datetime — дата/время
- enum_value_id (FK) — ссылка на значение перечисления

4. РАЗРАБОТКА БАЗЫ ДАННЫХ

Для хранения данных ... как и в предыдущих частях работы, была выбрана реляционная база данных PostgreSQL. Данное решение обеспечивает надёжное хранение структурированной информации, поддержку внешних ключей для обеспечения целостности ссылок и эффективную работу со связями между сущностями.

Рассмотрим в таблице 1 основные функции базы данных, обеспечивающие создание структуры хранения, добавление, изменение, удаление и перемещение элементов классификатора, а также управление продуктами и спецификациями.

Таблица 1 - Функции базы данных

Название функции	Назначение	Вход	Выход
create_parameter	Создание нового параметра	code, name, description, parameter_type, unit_id, enum_id	Новый параметр
list_parameters	Показ всех параметров	-	Все параметры
get_parameter	Получение конкретного параметра	parameter_id	Конкретный параметр
update_parameter	Обновление конкретного параметра	parameter_id, code, name, description, parameter_type, unit_id, enum_id	Конкретный параметр
delete_parameter	Удаление параметра	parameter_id	-

Пример заполнения таблиц представлен на рисунках 4, 5, 6, 7, 8.

123 id	AZ name	123 category_id	123 price	123 quantity	AZ description
2	Kingston Fury Beast Black KF432C16BBK2/16 DDR4 - 2x 8ГБ 3200МГц	8	6,990	12	Оперативная память для ПК
3	Gigabyte A520M K V2	9	7,990	5	Материнская плата для ПК
4	Intel Xeon Gold 6444Y 3.6ГГц	6	159,990	2	Серверный процессор
5	DDR4 Samsung M393A2K40EB3-16ГБ DIMM	5	11,990	10	Серверная оперативная память
6	Supermicro X11DPi-N	4	45,990	3	Серверная материнская плата
1	AMD Ryzen 7 5700X	7	14,990	7	16 ядер, 16 потоков
8	AMD Ryzen 5 7600X	11	25,990	12	6 ядер, 12 потоков, архитектура Zen 4
9	Intel Core i5-13600K	11	31,990	8	14 ядер, 20 потоков, Raptor Lake
10	NVIDIA GeForce RTX 4070	12	67,990	5	Видеокарта среднего-высокого класса
11	AMD Radeon RX 7800 XT	12	59,990	6	Видеокарта на архитектуре RDNA 3

Рисунок 4 — Пример заполнения таблицы products

123 id	123 category_id	123 parameter_id	123 priority	123 is_required	123 is_inherited	123 source_category_id	123 min_value	123 max_value
1	11	1	10	1	0	[NULL]	[NULL]	[NULL]
2	11	2	20	1	0	[NULL]	[NULL]	[NULL]
3	11	3	30	1	0	[NULL]	1	8
4	11	4	40	1	0	[NULL]	1	300
5	11	5	50	1	0	[NULL]	1	100
6	12	6	10	1	0	[NULL]	[NULL]	[NULL]
7	12	7	20	1	0	[NULL]	1	128
8	12	4	30	1	0	[NULL]	1	600

Рисунок 5 — Пример заполнения таблицы category_parameters

123 id	AZ full_name	AZ short_name	AZ description
1	нанометры	нм	Единица измерения длины, равная одной миллиардной части метра
2	Гигагерц	ГГц	Частота процессора
3	Ватт	Вт	Потребляемая мощность
4	Гигабайт	ГБ	Объем памяти

Рисунок 6 — Пример заполнения таблицы measurment_units

123 id	123 product_id	123 parameter_id	123 val_real	123 val_int	AZ val_str	val_datetime	123 enum_value_id
1	8	1	[NULL]	[NULL]	[NULL]	[NULL]	5
2	8	2	[NULL]	[NULL]	[NULL]	[NULL]	9
3	8	3	4.7	[NULL]	[NULL]	[NULL]	[NULL]
4	8	4	[NULL]	105	[NULL]	[NULL]	[NULL]
5	8	5	[NULL]	5	[NULL]	[NULL]	[NULL]
6	9	1	[NULL]	[NULL]	[NULL]	[NULL]	6
7	9	2	[NULL]	[NULL]	[NULL]	[NULL]	10
8	9	3	3.5	[NULL]	[NULL]	[NULL]	[NULL]
9	9	4	[NULL]	125	[NULL]	[NULL]	[NULL]
10	9	5	[NULL]	10	[NULL]	[NULL]	[NULL]
11	10	6	[NULL]	[NULL]	[NULL]	[NULL]	8
12	10	7	[NULL]	12	[NULL]	[NULL]	[NULL]
13	10	4	[NULL]	200	[NULL]	[NULL]	[NULL]
14	11	6	[NULL]	[NULL]	[NULL]	[NULL]	7
15	11	7	[NULL]	16	[NULL]	[NULL]	[NULL]
16	11	4	[NULL]	263	[NULL]	[NULL]	[NULL]

Рисунок 7 — Пример заполнения таблицы product_parameters_values

123 id	AZ code	AZ name	AZ description	AZ parameter_type	123 unit_id	123 enum_id
1	cpu_vendor	Производитель процессора	[NULL]	enum	[NULL]	4
2	socket	Сокет	[NULL]	enum	[NULL]	6
3	base_frequency	Базовая частота	[NULL]	real	2	[NULL]
4	tdp	TDP	[NULL]	integer	3	[NULL]
5	tech_process	Техпроцесс	[NULL]	integer	1	[NULL]
6	gpu_memory_type	Тип видеопамяти	[NULL]	enum	[NULL]	5
7	gpu_memory_size	Объем видеопамяти	[NULL]	integer	4	[NULL]

Рисунок 8 — Пример заполнения таблицы parameters

Функции, представленные выше в таблице, будут рассмотрены в ходе представления работы серверной части программы.

5. РАЗРАБОТКА ОСНОВНЫХ ПРОЦЕДУР

В таблице 2 представлены процедуры, через которые реализована работа с реестром.

Таблица 2 - Реализованные методы HTTP

Метод	URL	Описание
GET	/database/parameters	Получить список параметров
POST	/database/parameters	Создать параметр
GET	/database/parameters/{parameter_id}	Получить параметр
PATCH	/database/parameters/{parameter_id}	Изменить параметр
DELETE	/database/parameters/{parameter_id}	Удалить параметр
GET	/database/categories/{category_id}/parameters	Получить параметры категории
POST	/database/categories/{category_id}/parameters	Назначить параметр категории
PATCH	/database/category-parameters/{category_parameter_id}	Изменить назначение параметра категории
DELETE	/database/category-parameters/{category_parameter_id}	Удалить параметр из категории
POST	/database/category-parameters/{category_parameter_id}	Скопировать параметры родительской категории

Разработанные процедуры охватывают все необходимые операции для обслуживания перечислений в системе.

6. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

В данном разделе представлены тесты корректности работы реализованных API эндпоинтов

Посмотрим на получение списка всех параметров на рисунке 9.

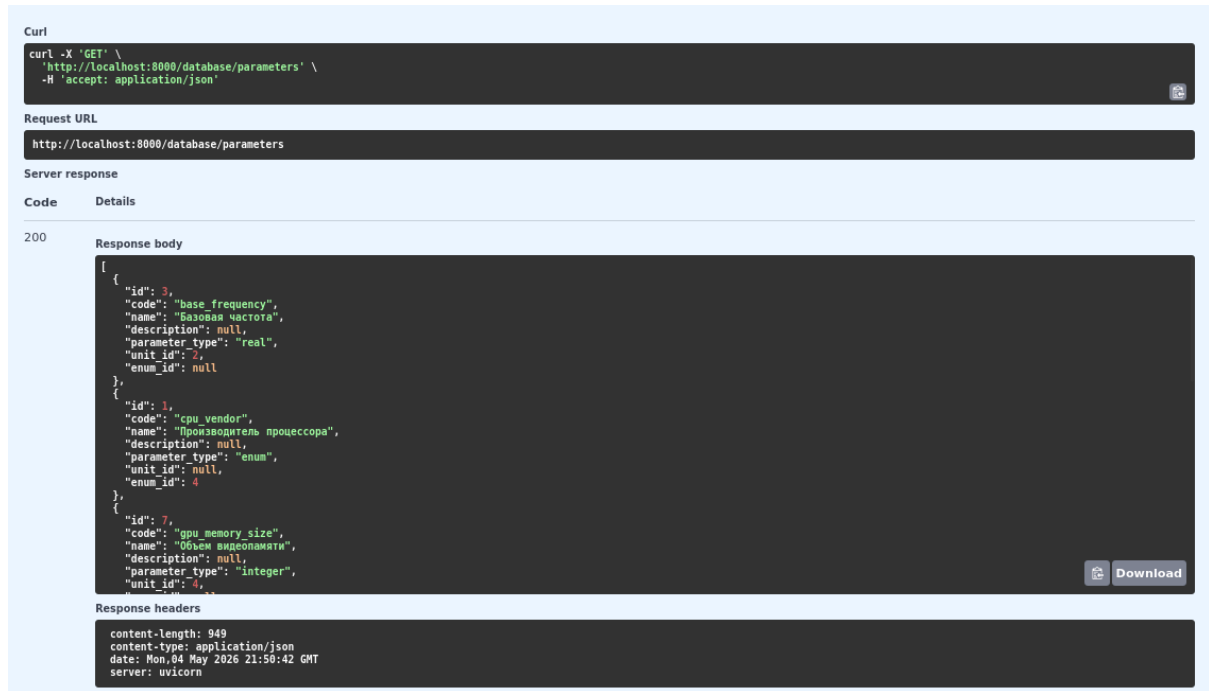


Рисунок 9 — Получение списка всех параметров

Демонстрация создания нового параметра представлена на рисунке 10.



Рисунок 10 — Создание нового параметра

На рисунках 11, 12, 13, 14 продолжаем тестирование реализованных функций

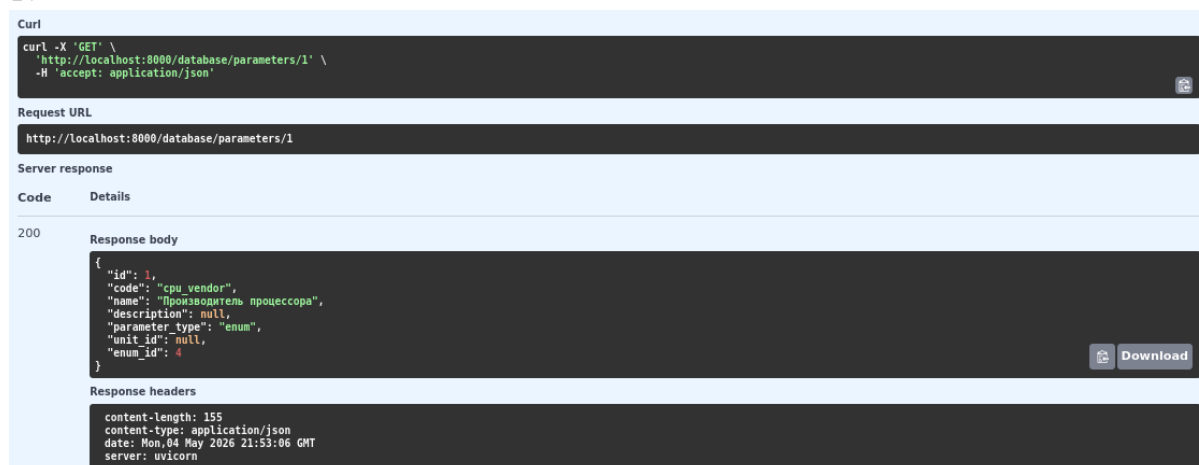


Рисунок 11 — Получение конкретного параметра

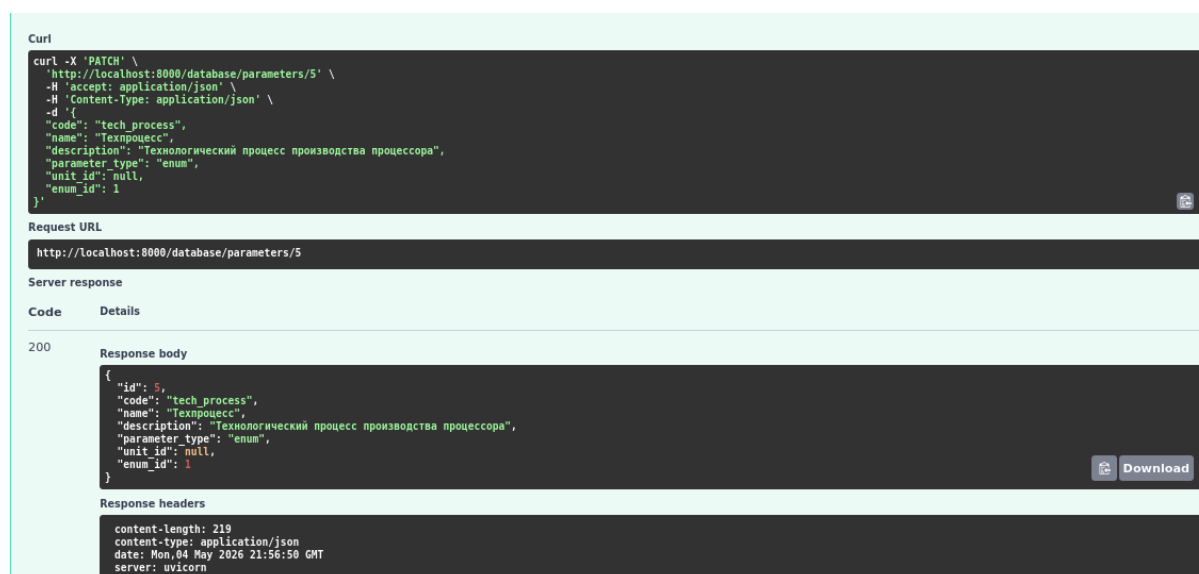


Рисунок 12 — Изменение конкретного параметра



Рисунок 13 — Неуспешное удаление элемента



Рисунок 14 — Успешное удаление элемента

По результатам проведённого тестирования можно сделать вывод, что реализованные процедуры работы с перечислениями функционируют корректно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы была разработана подсистема справочников, расширяющая ранее созданный классификатор изделий.

Проведен анализ предметной области, позволивший определить требования к системе. На его основе разработаны объектная модель и модель хранения данных, обеспечивающие гибкое управление параметрами изделий.

Реализованные процедуры позволяют выполнять полный набор операций по работе со справочниками, включая создание, редактирование и поиск данных.

Результаты тестирования подтвердили корректность функционирования системы и её соответствие предъявляемым требованиям.

Разработанное решение обеспечивает расширяемость и гибкость информационной системы, позволяя адаптировать её под различные предметные области без изменения программного кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ Р 7.0.97–2025. Национальный стандарт Российской Федерации. Система стандартов по информации, библиотечному и издательскому делу. Организационно-распорядительная документация. Требования к оформлению документов. М.: Стандарт, 2025.
2. Каталог интернет-магазина техники // Ситилинк URL: <https://www.citilink.ru/catalog/komplektuyushchie-dlya-pk/?ref=mainmenu> (дата обращения: 20.04.2026).
3. Каталог интернет-магазина техники // DNS URL: <https://www.dns-shop.ru/catalog/17aa529516404e77/komplektuyushchie-dla-servera/> (дата обращения 20.04.2026).